S.No: 1	Exp. Name: ***Display hello world message***	**Date: 2025-11-20**

Aim:
Write a C program to display H**ello World** message.

Source Code:

hello.c

```
#include<stdio.h>
void main()
{
printf("Hello World\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Hello World

S.No: 2	Exp. Name: ***Scan all data type variables and display them***	**Date: 2025-12-04**

Aim:

Write a C program to scan all data type variables(int , float, char, double) as input and print them as output.

Input Format:

- First Line: An integer, entered after the prompt "integer: ".
- Second Line: A floating-point number, entered after the prompt "floating-point number: ".
- Third Line: A character, entered after the prompt "character: ".
- Fourth Line: A double-precision floating-point number, entered after the prompt "double: ".

Output Format:

- First Line: A message "You entered:".
- Second Line: The integer entered, in the format "Integer: <integerVar>".
- Third Line: The floating-point number entered, formatted to six decimal places, in the format "Float: <floatVar>".
- Fourth Line: The character entered, in the format "Character: <charVar>".
- Fifth Line: The double-precision floating-point number entered, formatted to six decimal places, in the format "Double: <doubleVar>".

Note: Add a space before %c to skip any whitespace (spaces, tabs, or newlines) when reading a character.

Source Code:

scan.c

```
#include<stdio.h>
void main()
{
int a;
float b;
char c;
double d;

printf("integer: ");
scanf("%d",&a);
printf("floating-point number: ");
scanf("%f",&b);
printf("character: ");
scanf(" %c",&c);
printf("double: ");
scanf("%lf",&d);
printf("You entered:\n");
printf("Integer: %d\n",a);
printf("Float: %f\n",b);
printf("Character: %c\n",c);
printf("Double: %lf\n",d);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
integer:
9
floating-point number:
12.0254
character:
C
double:
12.02543124
You entered:

Integer: 9
Float: 12.025400
Character: C
Double: 12.025431

Test Case - 2
User Output
integer:
-10
floating-point number:
12.2546
character:
T
double:
12.6789678
You entered:
Integer: -10
Float: 12.254600
Character: T
Double: 12.678968

S.No: 3	Exp. Name: ***Arithmetic Operations***	**Date: 2025-12-04**

Aim:

Write a C program to perform arithmetic operations like **+, -, *, /,** and **%** on two input variables $num1$ and $num2$.

Input Format:

- The first line of input is an integer representing the value for the first number, $num1$.
- The second line of input is an integer representing the value of the second number, $num2$.

Output Format:

- The program prints the integers that represent the results of addition, subtraction, multiplication, division, and modulus, each on a new line.

Note:

- Assuming that he value of $num2$ is always greater than 0.
- Add a new line character (**\n**) at the end of the output.

Source Code:

arithmeticOperations.c

```
#include <stdio.h>
void main()
{
int a,b;
scanf("%d %d",&a,&b);
printf("%d\n",a+b);
printf("%d\n",a-b);
printf("%d\n",a*b);
printf("%d\n",a/b);
printf("%d\n",a%b);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
9
8
17
1
72
1
1

Test Case - 2
User Output
1000
2
1002
998
2000
500
0

S.No: 4	Exp. Name: ***Write a C program to find Sum and Average of three numbers***	**Date: 2025-12-04**

Aim:

Write a program to find the `sum` and `average` of the three given integers.

Note: Use the **printf()** function with a **newline** character (`\n`) at the end.

Source Code:

Program314.c

```
#include<stdio.h>
 void main()
{

int a,b,c,sum;
float average;
printf("Enter three integers : ");
scanf("%d %d %d",&a,&b,&c);
sum=a+b+c;
printf("Sum of %d, %d and %d : %d\n",a,b,c,sum);
average=sum/3.0;
printf("Average of %d, %d and %d : %f\n",a,b,c,average);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter three integers :
121 34 56

Sum of 121, 34 and 56 : 211
Average of 121, 34 and 56 : 70.333336

Test Case - 2
User Output
Enter three integers :
5 8 3
Sum of 5, 8 and 3 : 16
Average of 5, 8 and 3 : 5.333333

Test Case - 3
User Output
Enter three integers :
-1 5 -6
Sum of -1, 5 and -6 : -2
Average of -1, 5 and -6 : -0.666667

S.No: 5	Exp. Name: ***Temperature conversions from Centigrade to Fahrenheit and vice versa.***	**Date: 2025-12-04**

Aim:
Write a C program to perform temperature conversions from Celsius to Fahrenheit

Formula:
$Fahrenheit = (Celsius * 9/5) + 32$

Input Format:
- Input a single floating-point number representing the temperature in Celsius to be converted to Fahrenheit.

Output Format:
- The program will print the temperature in both Celsius and Fahrenheit, formatted to two decimal places.

Note: Some parts of the code are given, fill in the missing code.

Source Code:

temperature.c

```
#include <stdio.h>

int main() {
float c,f;
scanf("%f",&c);
f=(c*9/5.0)+32;



        printf("%.2f Celsius = %.2f Fahrenheit\n",c,f);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

37.5
37.50 Celsius = 99.50 Fahrenheit

Test Case - 2
User Output
-20
-20.00 Celsius = -4.00 Fahrenheit

S.No: 6	Exp. Name: ***Problem Solving***	Date: 2025-12-26

Aim:

Write a program to calculate the `simple interest` by reading **principle amount**, **rate of interest** and **time**.

At the time of execution, the program should print the message on the console as:

```
Enter principle amount, rate of interest, time of loan :
```

For example, if the user gives the **input** as:

```
Enter principle amount, rate of interest, time of loan : 23456.78
3.5 2.5
```

then the program should **print** the result as:

```
Simple Interest = 2052.468018
```

Note: Do use the **printf()** function and ensure that there is a `'\n'` at the end after print the result.

Source Code:

Program3.c

```
#include <stdio.h>
void main ()
{
        float p,r,t;
        float si;
        printf("Enter principle amount, rate of interest, time of loan
: ");
        scanf("%f %f %f",&p,&r,&t);
        si=(p*r*t)/100;
        printf ("Simple Interest = %.6f\n",si);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter principle amount, rate of interest, time of loan :
2500 5 2
Simple Interest = 250.000000

S.No: 7	Exp. Name: ***Calculate the square root of an integer***	**Date: 2025-12-26**

Aim:
Write a program that prompts the user to enter an integer and calculates its square root.

Note: Print the result up to 3 decimal places.

Input format:
The program takes an integer as input with the print statement **"Enter an integer: "** followed by the integer.

Output format:
The output is the floating point value formatted to three decimals that represents the square root value of the user-given integer.

Hint: You can use **math** library to perform mathematical operations.

Instruction: During writing your code, please follow the input and output layout as mentioned in the sample test case.

Source Code:

squareRoot.c

```
#include<stdio.h>
#include<math.h>
void main()
{
        int x;
        float y;
        printf ("Enter an integer: ");
        scanf("%d",&x);
        y=sqrt(x);
        printf("Square root: %.3f\n",y);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an integer:

2
Square root: 1.414

Test Case - 2
User Output
Enter an integer:
4
Square root: 2.000

S.No: 8	Exp. Name: ***Calculate simple interest and compound interest***	**Date: 2025-12-26**

Aim:

Write a C program to calculate the simple interest and compound interest by reading **principal amount**, **rate of interest** and **time**.

Note: Use the **printf()** function and ensure that the character `'\n'` is printed at the end of the result.

Input Format:

- Prompt the user to enter the values of P, R, T separated by space

Output Format:

- The First line of output represents the value of Simple Interest (SI)
- The Second line of output represents the value of Compound Interest (CI)

Example:

Input:

5000 7 5

Output:

```
SI= 1750.000000
CI= 2012.760376
```

The formula to find simple interest is SI = $\frac{(principal*rate*time)}{100}$

The formula to find compound interest is CI = $principal * pow(1 + \left(\frac{rate}{100}\right),\ time) - principal$

Note: Use float data type for all the involved variables.

Source Code:

Program315.c

```
#include<stdio.h>
#include<math.h>
void main()
{
        float p,r,t;
        float si,ci;
        scanf("%f %f %f",&p,&r,&t);
        si = (p*r*t)/100;
        printf("SI= %f\n",si);
        ci = p*pow(1+(r/100),t)-p;
        printf("CI= %f\n",ci);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5000 7 5
SI= 1750.000000
CI= 2012.760376

Test Case - 2
User Output
10000 4.5 3.5
SI= 1575.000000
CI= 1665.596558

S.No: 9	Exp. Name: ***Area of a triangle using heron's formula***	**Date: 2025-12-27**

Aim:

Write a program to find the `area` of a **triangle** using Heron's formula.

During execution, the program should print the following message on the console:

```
sides:
```

For example, if the user gives the following as **input** (input is positive floating decimal point numbers):

```
sides: 2.3 2.4 2.5
```

Then the program should **print** the result round off upto 2 decimal places as:

```
area: 2.49
```

Instruction: Your input and output layout must match with the sample test cases **(values as well as text strings)**.

The area of a triangle is given by `Area = √ p(p – a)(p – b)(p – c)`, where `p` is half of the perimeter, or `(a + b + c) / 2`. Let a,b,c be the lengths of the sides of the given triangle.

Hint: Use `sqrt` function defined in `math.h` header file

Source Code:

Program313.c

```
#include<stdio.h>
#include<math.h>
void main()
{
        float a,b,c,s,area;
        printf("sides: ");
        scanf("%f%f%f",&a,&b,&c);
        s=(a+b+c)/2;
        area=sqrt(s*(s-a)*(s-b)*(s-c));
        printf("area: %.2f\n",area);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
sides:
2.3 2.4 2.5
area: 2.49

Test Case - 2
User Output
sides:
2.6 2.7 2.8
area: 3.15

S.No: 10	Exp. Name: ***Distance travelled by an object***	**Date: 2025-12-27**

Aim:

Write a program to find the `distance` travelled by an object.

Sample Input and Output:

```
Enter the acceleration value : 2.5
Enter the initial velocity : 5.7
Enter the time taken : 20
Distance travelled : 614.000000
```

Note - 1: Use the formula to find distance, `distance = ut + (1/2) at²`.

Note: Use the **printf()** function with a **newline** character (`\n`) at the end.

Source Code:

DistanceTravelled.c

```
#include<stdio.h>
int main(){
        float a,i,t,d;
        printf("Enter the acceleration value : ");
        scanf("%f",&a);
        printf("Enter the initial velocity : ");
        scanf("%f",&i);
        printf("Enter the time taken : ");
        scanf("%f",&t);
        d=(i*t+(0.5)*a*t*t);
        printf("Distance travelled : %f\n",d);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the acceleration value :
4

Enter the initial velocity :
5
Enter the time taken :
6
Distance travelled : 102.000000

Test Case - 2
User Output
Enter the acceleration value :
5
Enter the initial velocity :
0
Enter the time taken :
10
Distance travelled : 250.000000

Test Case - 3
User Output
Enter the acceleration value :
2.5
Enter the initial velocity :
5.7
Enter the time taken :
20
Distance travelled : 614.000000

Test Case - 4
User Output
Enter the acceleration value :
50
Enter the initial velocity :
34.67
Enter the time taken :
6
Distance travelled : 1108.020020

Test Case - 5
User Output
Enter the acceleration value :
125.6
Enter the initial velocity :
45.8
Enter the time taken :
4
Distance travelled : 1188.000000

S.No: 11	Exp. Name: ***Evaluate the expressions***	Date: 2025-12-27

Aim:

Write a C program to evaluate the following expressions.

a. A+B*C+(D*E) + F*G

b. A/B*C-B+A*D/3

c. A+++B---A

d. J= (i++) + (++i)

Note: consider expression as A++ + ++B - --A

Source Code:

evaluate.c

```
#include<stdio.h>
int main(){
        int A,B,C,D,E,F,G,i,j;
        printf("Enter values for A, B, C, D, E, F, G, i: ");
        scanf("%d%d%d%d%d%d%d%d",&A,&B,&C,&D,&E,&F,&G,&i);
        int a=A+B*C+(D*E)+F*G;
        printf("a.A+B*C+(D*E) + F*G = %d\n" , a);
        int b=A/B*C-B+A*D/3;
        printf("b.A/B*C-B+A*D/3 = %d\n" , b);
        int c=A++ + ++B- --A;
        printf("c.A+++B---A = %d\n",c);
        int d=(i++)+(++i);
        printf("d.J = (i++) + (++i) = %d\n",d);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter values for A, B, C, D, E, F, G, i:
1 2 3 4 5 6 7 8
a.A+B*C+(D*E) + F*G = 69
b.A/B*C-B+A*D/3 = -1
c.A+++B---A = 3
d.J = (i++) + (++i) = 18

Test Case - 2
User Output
Enter values for A, B, C, D, E, F, G, i:
10 20 60 30 40 4 6 1
a.A+B*C+(D*E) + F*G = 2434
b.A/B*C-B+A*D/3 = 80
c.A+++B---A = 21
d.J = (i++) + (++i) = 4

S.No: 12	Exp. Name: ***Greatest of three numbers using a conditional operator***	**Date: 2025-12-27**

Aim:

Write a C program to display the greatest of three numbers using a conditional operator (ternary operator).

Input Format:

- The first three lines of input each contain one integer representing $num1$, $num2$, and $num3$, respectively.

Output Format:

- The output displays the greatest of the three entered integers.

Note: Refer to sample test cases for better understanding.

Source Code:

greatest.c

```
#include <stdio.h>

int main() {
        int num1, num2, num3, greatest;

    // Take input from user
        printf("num1: ");
    scanf("%d",&num1);

        printf("num2: ");
        scanf("%d",&num2);

        printf("num3: ");
        scanf("%d",&num3);

    // Find the greatest number using the conditional operator
        greatest = (num1>num2 && num1>num3)?num1:(num2>num3 ?
num2:num3);

    // Display the greatest number
        printf("Greatest number: %d\n",greatest);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
num1:
8
num2:
9
num3:
90
Greatest number: 90

Test Case - 2
User Output
num1:
5
num2:
45
num3:
6
Greatest number: 45

S.No: 13	Exp. Name: ***Write a C program to Total and Average of 5 subjects marks***	**Date: 2025-12-27**

Aim:

Write a program to take marks of `5` subjects in **integers**, and find the `total`, `average` in **float**.

Sample Input and Output:

```
Enter 5 subjects marks : 55 56 57 54 55
Total marks : 277.000000
Average marks : 55.400002
```

Note: Use the **printf()** function with a **newline** character (`\n`) to print the output at the end.

Source Code:

TotalAndAvg.c

```
#include<stdio.h>
int main(){
        int marks[5];
        int total=0;
        float average;
        printf("Enter 5 subjects marks : ");
        for(int i=0;i<5;i++){
                scanf("%d",&marks[i]);
                total+=marks[i];
        }
        average=total/5.0;
        printf("Total marks : %.6f\n",(float)total);
        printf("Average marks : %.6f\n",average);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter 5 subjects marks :

45 67 89 57 49
Total marks : 307.000000
Average marks : 61.400002

Test Case - 2
User Output
Enter 5 subjects marks :
55 56 57 54 55
Total marks : 277.000000
Average marks : 55.400002

Test Case - 3
User Output
Enter 5 subjects marks :
90 97 95 92 91
Total marks : 465.000000
Average marks : 93.000000

Test Case - 4
User Output
Enter 5 subjects marks :
20 30 66 77 44
Total marks : 237.000000
Average marks : 47.400002

Test Case - 5
User Output
Enter 5 subjects marks :
56 78 88 79 64
Total marks : 365.000000
Average marks : 73.000000

Test Case - 6

User Output
Enter 5 subjects marks :
44 35 67 49 51
Total marks : 246.000000
Average marks : 49.200001

S.No: 14	Exp. Name: ***Write a Program to find the Max and Min of Four numbers***	**Date: 2025-12-27**

Aim:

Write a program to find the `max` and `min` of **four** numbers.

Sample Input and Output :

```
Enter 4 numbers : 9 8 5 2
Max value : 9
Min value : 2
```

Note: Use the **printf()** function with a **newline** character (`\n`) to print the output at the end.

Source Code:

MinandMaxOf4.c

```
#include<stdio.h>
int main()
{
        int num1,num2,num3,num4;
        int max,min;
        printf("Enter 4 numbers : ");
        scanf("%d%d%d%d",&num1, &num2,&num3,&num4);
        max=num1;
        min=num1;
        if(num2>max)max=num2;
        if(num3>max)max=num3;
        if(num4>max)max=num4;
        if(num2<min)min=num2;
        if(num3<min)min=num3;
        if(num4<min)min=num4;
        printf("Max value : %d\n",max);
        printf("Min value : %d\n",min);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter 4 numbers :
9 8 5 2
Max value : 9
Min value : 2

Test Case - 2
User Output
Enter 4 numbers :
112 245 167 321
Max value : 321
Min value : 112

Test Case - 3
User Output
Enter 4 numbers :
110 103 113 109
Max value : 113
Min value : 103

Test Case - 4
User Output
Enter 4 numbers :
-34 -35 -24 -67
Max value : -24
Min value : -67

Test Case - 5
User Output
Enter 4 numbers :
24 28 34 16
Max value : 34

Min value : 16

Test Case - 6
User Output
Enter 4 numbers :
564 547 574 563
Max value : 574
Min value : 547

S.No: 15	Exp. Name: ***Find out the electricity bill charges***	**Date: 2025-12-27**

Aim:

An electricity board charges the following rates for the use of electricity:

- If units are less than or equal to 200, then the charge is calculated as 80 paise per unit.
- If units are less than or equal to 300, then the charge is calculated as 90 paise per unit.
- If units are beyond 300, then the charge is calculated as 1 Rupee per unit.

All users are charged a minimum of Rs. 100 as a meter charge even though the amount calculated is less than Rs. 100.

If the total amount charged is greater than Rs. 400, then an additional surcharge of 15% of the total amount is charged.

Write a C program to read the name of the user, and the number of units consumed and print out the charges as shown in the sample test cases.

Note: Print the amount charged up to 2 decimal places (actual amount, surcharges, amount to be paid).

Source Code:

`electricityBillCharges.c`

```c
#include<stdio.h>
int main(){
        char name[50];
        int units;
        float amount, surcharge=0.0, total;
        printf("Enter customer name: ");
        scanf("%s",name);
        printf("Units consumed: ");
        scanf("%d",&units);
        if(units<=200)
                amount=units*0.80;
        else if(units<=300)
                amount=units*0.90;
        else
                amount=units*1.00;
        printf("Customer name: %s\n",name);
        printf("Units consumed: %d\n",units);
        printf("Amount charged: %.2f\n",amount);
        if(amount<100)
                amount=100;
        if(amount>400)
                surcharge=amount*0.15;
        total=amount+surcharge;
        printf("Surcharges: %.2f\n",surcharge);
        printf("Amount to be paid: %.2f\n",total);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter customer name:
John
Units consumed:
78
Customer name: John
Units consumed: 78
Amount charged: 62.40
Surcharges: 0.00
Amount to be paid: 100.00

Test Case - 2
User Output
Enter customer name:
Rosy
Units consumed:
325
Customer name: Rosy
Units consumed: 325
Amount charged: 325.00
Surcharges: 0.00
Amount to be paid: 325.00

Test Case - 3
User Output
Enter customer name:
Amar
Units consumed:
801
Customer name: Amar
Units consumed: 801
Amount charged: 801.00
Surcharges: 120.15
Amount to be paid: 921.15

Test Case - 4
User Output
Enter customer name:
Raman
Units consumed:
300
Customer name: Raman
Units consumed: 300
Amount charged: 270.00
Surcharges: 0.00
Amount to be paid: 270.00

S.No: 16	Exp. Name: ***C program to find roots and nature of quadratic equation.***	**Date: 2025-12-27**

Aim:

Write a C program to find the roots of a quadratic equation, given its coefficients.

Sample Input and Output:

```
Enter coefficients a, b and c: 3 7 9
root1 = -1.17+1.28i and root2 = -1.17-1.28i
```

Source Code:

quad.c

```
#include<stdio.h>
#include<math.h>
int main(){
        float a,b,c,D,real,img;
        printf("Enter coefficients a, b and c: ");
        scanf("%f %f %f",&a, &b, &c);
        D=(b*b)-(4*a*c);
        if(a==0){
                printf("Not a quadratic equation");
        }
        else if(D>0){
                float root1=(-b+sqrt(D))/(2*a);
                float root2=(-b+sqrt(D))/(2*a);
                printf("Roots are real and distinct.\n");
                printf("root1=%.2f and root2=%.2f",root1,root2);
        }
        else if(D==0){
                float root=-b/(2*a);
                printf("Roots sre equal and real.\n");
                printf("root1=root2=%.2f",root);
        }
        else{
                real=-b/(2*a);
                img=sqrt(-D)/(2*a);
                printf("root1 = %.2f+%.2fi and ",real,img);
                printf("root2 = %.2f-%.2fi",real,img);
        }
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter coefficients a, b and c:
3 7 9
root1 = -1.17+1.28i and root2 = -1.17-1.28i

Test Case - 2
User Output
Enter coefficients a, b and c:
8 8 6
root1 = -0.50+0.71i and root2 = -0.50-0.71i

S.No: 17	Exp. Name: ***Simulate a basic calculator***	**Date: 2025-12-27**

Aim:

Write a program to perform basic calculator operations **[+, -, *, /]** of two integers **a** and **b** using switch statement.

Constraints:

- 10^{-4} <= a,b = 10^{4}
- operations allowed are +, -, *, /
- "/" divisibility will perform integer division operation.

Input Format: The first line of the input consists of an integer which corresponds to **a**, character which corresponds to the **operator** and an integer which corresponds to **b**.

Output format: Output consists of result after performing mentioned operation (a operation b).

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

Source Code:

calculator.c

```
#include<stdio.h>
#include<math.h>
void main(){
        int a,b;
        char op;
        scanf("%d%c%d",&a,&op,&b);
        switch(op){
                case '+':
                printf("%d\n",a+b);
                break;
                case '-':
                printf("%d\n",a-b);
                break;
                case '*':
                printf("%d\n",a*b);
                break;
                case '/':
                printf("%d\n",a/b);
                break;
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
36-31
5

Test Case - 2
User Output
89/45
1

Test Case - 3
User Output
13+1000
1013

Test Case - 4
User Output
25*500
12500

S.No: 18	Exp. Name: ***Write a program to find leap year or not***	**Date: 2025-12-27**

Aim:

Lucy is celebrating her 15th birthday. Her father promised her that he will buy her a new computer on her birthday if she solves the question asked by him.

He asks Lucy to find whether the year on which she had born is **leap year or not**.

Help her to solve this puzzle so that she celebrates her birthday happily. If her birth year is 2016 and it is a leap year display 2016 is a leap year.? Else display 2016 is not a leap year and check with other leap year conditions.

Source Code:

leapYear.c

```
#include<stdio.h>
#include<math.h>
void main(){
        int year;
        scanf("%d",&year);
        if((year%4==0&&year%100!=0)||year%400==0)
        {
                printf("%d is a leap year\n",year);
        }
        else{
                printf("%d is not a leap year\n",year);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1900
1900 is not a leap year

Test Case - 2
User Output

2004
2004 is a leap year

Test Case - 3
User Output
1995
1995 is not a leap year

S.No: 19	Exp. Name: ***Factorial of a given number***	Date: 2025-12-27

Aim:

Write a C program to find the factorial of a given number

Source Code:

factorialOfInt.c

```
#include<stdio.h>
void main(){
        int a,f=1,i;
        printf("Integer: ");
        scanf("%d",&a);
        for(i=1;i<=a;i++){
                f=f*i;
        }
        printf("Factorial: %d\n",f);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Integer:
5
Factorial: 120

Test Case - 2
User Output
Integer:
4
Factorial: 24

S.No: 20	Exp. Name: ***C program to determine whether a given number is prime or not.***	**Date: 2025-12-27**

Aim:

Write the C program to determine whether a given number is prime or not.

Source Code:

Prime.c

```
#include<stdio.h>
#include<math.h>
void main(){
int  n,i,count=0;
printf("Enter a number: ");
scanf("%d",&n);
for(i=0;i<=n;i++){
        if(n%i==0)
                count++;
}
if(count==2)
        printf("%d is a prime number\n",n);
else
        printf("%d is not a prime number\n",n);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a number:
9
9 is not a prime number

Test Case - 2
User Output
Enter a number:
11

```
11 is a prime number
```

S.No: 21	Exp. Name: ***Compute sine and cos series using taylor series***	**Date: 2025-12-27**

Aim:

Write a C program to compute the sine and cosine series using the Taylor series.

Taylor series:

$\sin x = x - (x^3/3!) + (x^5/5!) - (x^7/7!) + ...$

$\cos x = 1 - (x^2/2!) + (x^4/4!) - (x^6/6!) + ...$

Note: Print the result up to 4 decimal places. Use the **double** data type for all variables except for the number of terms in the series, which should be an integer. Additionally, initialize the variables that will store the results of the sine and cosine series to **0.0** at the beginning.

Source Code:

taylor.c

```
#include<stdio.h>
#include<math.h>
int main(){
        double x,sinx=0.0,cosx=0.0,term;
        int n,i;
        printf("angle in radians: ");
        scanf("%lf",&x);
        printf("number of terms in the series: ");
        scanf("%d",&n);
        for(i=0;i<n;i++){
                term=pow(-1,i)*pow(x,2*i+1)/tgamma(2*i+2);
                sinx+=term;
        }
        for(int i=0;i<n;i++){
                term=pow(-1,i)*pow(x,2*i)/tgamma(2*i+1);
                cosx+=term;
        }
        printf("Sine = %.4lf\n",sinx);
        printf("Cosine = %.4lf\n",cosx);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
angle in radians:
0.5
number of terms in the series:
3
Sine = 0.4794
Cosine = 0.8776

Test Case - 2
User Output
angle in radians:
0.6
number of terms in the series:
5
Sine = 0.5646
Cosine = 0.8253

S.No: 22	Exp. Name: ***Check given number is palindrome or not***	**Date: 2025-12-27**

Aim:

Write an C program to check given number is palindrome or not

Input Format:

- Single Line: An integer value representing the number to be checked for palindrome status.

Output Format:

- Single Line: A message indicating whether the number is a palindrome or not. The format of the message will be:
- "**[number] is a palindrome.**" if the number is a palindrome.
- "**[number] is not a palindrome.**" if the number is not a palindrome.

Source Code:

palindrome.c

```
#include<stdio.h>
void main()
{
        int n,rem,rev=0,dup;
        scanf("%d",&n);
        dup=n;
        while(n>0)
                {
                        rem=n%10;
                        rev=rev*10+rem;
                        n=n/10;
}
        if(dup==rev)
                printf("%d is a palindrome.", dup);
        else
                printf("%d is not a palindrome.",dup);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

121
121 is a palindrome.

Test Case - 2
User Output
143
143 is not a palindrome.

S.No: 23	Exp. Name: ***Pyramid with numbers***	Date: 2025-12-27

Aim:

Write a program to print a `pyramid` of **numbers** separated by spaces for the given number of rows.

At the time of execution, the program should print the message on the console as:

```
Enter number of rows :
```

For example, if the user gives the **input** as :

```
Enter number of rows : 3
```

then the program should **print** the result as:

```
  1
 1 2
1 2 3
```

Source Code:

PyramidDemo15.c

```
#include <stdio.h>
void main() {
        int n, i, j, s;
        printf("Enter number of rows : ");
        scanf("%d", &n);
        // Fill the missing code
        for(int i=1;i<=n;i++){
                for(int s=1;s<=n-i;s++)
                        printf(" ");
                for(int j=1;j<=i;j++)
                        printf("%d ",j);
                printf("\n");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of rows :
3
1
1 2
1 2 3

Test Case - 2
User Output
Enter number of rows :
6
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6

Test Case - 3
User Output
Enter number of rows :
8
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

S.No: 24	Exp. Name: ***Minimum and maximum in an array of integers.***	**Date: 2025-12-27**

Aim:

Write a C program to find the **minimum** and **maximum** in an array of integers.

Source Code:

ArrayElements.c

```
#include <stdio.h>
void main() {
        int arr[20], number, min = 0, max = 0;
        scanf("%d", &number);
        printf("Elements: ");
        for (int i = 0; i < number; i++) {
                scanf("%d", &arr[i]);
        }
        /* Write your logic here to find the maximum and minimum in the
given integer array*/
        min=max=arr[0];
        for(int i=1;i<number;i++){
                if(arr[i]<min)
                        min=arr[i];
                if(arr[i]>max)
                        max=arr[i];
        }
        printf("Min an Max: %d and %d", min, max);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
Elements:
4 9 6 8 2
Min an Max: 2 and 9

Test Case - 2

User Output
1
Elements:
216
Min an Max: 216 and 216

S.No: 25	Exp. Name: ***Search for an element using Linear search***	**Date: 2025-12-27**

Aim:

Write a C program to check whether a given element is present in an array of elements using linear search. The program should prompt the user to enter the size of the array, the elements of the array, and the element to search for. It should then output whether the element is found, and if so, at which position in the array.

Input Format:

- Prompt the user to enter an integer n representing the size of the array ($1 \leq n \leq 20$) as follows:
- In the next line, enter n integers representing the elements of the array separated by a space.
- The next line should input an integer key representing the element to search for.

Output Format:

- If the element is found, output: "Found at position X" (where X is the 0-based index of the found element).
- If the element is not found, output: "Y is not found" (where Y is the key that was searched for).

Note: Refer to the visible test cases to strictly match with the input/output layout.

Source Code:

SearchEle.c

```
#include<stdio.h>
int main(){
        int i,n,key,found=0;
        printf("Enter size: ");
        scanf("%d",&n);
        int arr[n];
        printf("Enter %d element: ",n);
        for(int i=0;i<n;i++)
                scanf("%d",&arr[i]);
        printf("Enter search element: ");
        scanf("%d",&key);
        for(int i=0;i<n;i++)
                if(arr[i]==key){
                        printf("Found at position %d\n",i);
                        found=1;
                        break;
                }
                if(!found){
                        printf("%d is not found\n",key);
                }
        }
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter size:
6
Enter 6 element:
2 4 8 1 3 5
Enter search element:
6
6 is not found

Test Case - 2
User Output
Enter size:
6
Enter 6 element:

2 4 8 1 3 5
Enter search element:
2
Found at position 0

Test Case - 3
User Output
Enter size:
6
Enter 6 element:
2 4 8 1 3 5
Enter search element:
9
9 is not found

S.No: 26	Exp. Name: ***C program to reverse an array.***	**Date: 2025-12-27**

Aim:

Write a C program to reverse the elements an array of integers.

Source Code:

reverseArray.c

```
#include<stdio.h>
int main(){
        int n;
        printf("Enter no of elements: ");
        scanf("%d",&n);
        int arr[n];
        printf("Enter elements: ");
        for(int i=0;i<n;i++){
                scanf("%d",&arr[i]);
        }
        printf("The reversed array: ");
        for(int i=n-1;i>=0;i--){
                printf("%d ",arr[i]);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of elements:
5
Enter elements:
3 4 1 2 4
The reversed array: 4 2 1 4 3

Test Case - 2
User Output
Enter no of elements:

8
Enter elements:
2 5 1 77 33 88 2 9
The reversed array: 9 2 88 33 77 1 5 2

S.No: 27	Exp. Name: ***2's complement of a given Binary number***	**Date: 2025-12-27**

Aim:
Write a **C** program to find 2's complement of a given binary number.

Note: The binary input should be separated by a **space**.
Source Code:

twosComplement.c

```
#include<stdio.h>
int main(){
        int a[50],n,i;
        printf("Enter size: ");
        scanf("%d",&n);
        printf("Enter %d bit binary number: ",n);
        for(int i=0;i<n;i++){
                scanf("%d",&a[i]);
        }
        for(int i=0;i<n;i++){
                a[i]=(a[i]==0)?1:0;
        }
        int carry=1;
        for(int i=n-1;i>=0;i--){
                int sum=a[i]+carry;
                a[i]=sum%2;
                carry=sum/2;
                if(carry==0)
                        break;
        }
        printf("2's complement: ");
        for(int i=0;i<n;i++){
                printf("%d ",a[i]);
        }
        printf("\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter size:
5
Enter 5 bit binary number:
1 0 0 1 0
2's complement: 0 1 1 1 0

Test Case - 2
User Output
Enter size:
6
Enter 6 bit binary number:
1 0 0 0 1 1
2's complement: 0 1 1 1 0 1

S.No: 28	Exp. Name: ***Remove Duplicates from an Array***	**Date: 2025-12-27**

Aim:

Write a C program to read n integers into an array and remove all duplicate elements. The program should output only the unique elements in the order they first appeared.

Input Format:

- The first line prompts "Enter size: " followed by an integer n, representing the size of the array.
- The second line prompts "Enter <n> elements: " followed by n space-separated integers representing the elements of the array.

Output Format:

- Prompt "After eliminating duplicates: " followed by a single line containing the unique elements of the array after removing duplicates
- Elements should be space-separated and in the order they first appeared in the input.

Note:

- Refer to the sample test cases to strictly match the input/output format.

Source Code:

eliminateDuplicates.c

```c
#include<stdio.h>
int main(){
        int n,i,j,a[100],unique[100],k=0;
        printf("Enter size: ");
        scanf("%d", &n);
        printf("Enter %d elements: ",n);
        for(int i=0;i<n;i++){
                scanf("%d",&a[i]);
        }
        for(int i=0;i<n;i++){
                int found=0;
                for(int j=0;j<k;j++){
                        if(unique[j]==a[i]){
                                found=1;
                                break;
                        }
                }
                if(!found){
                        unique[k++]=a[i];
                }
        }
        printf("After eliminating duplicates: ");
        for(int i=0;i<k;i++){
                printf("%d ",unique[i]);
        }
        printf("\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter size:
5
Enter 5 elements:
1 2 1 2 3
After eliminating duplicates: 1 2 3

Test Case - 2

User Output
Enter size:
5
Enter 5 elements:
11 13 11 12 13
After eliminating duplicates: 11 13 12

S.No: 29	Exp. Name: ***Addition of Two Matrices***	Date: 2025-12-27

Aim:

Write a C program to perform the addition of two matrices, each with the size $m \times n$.

Input Format:

- The program will first display "Enter no of rows, columns: "and read two integers representing the number of rows and columns, separated by a space.
- Then it will display "Elements of matrix 1: "and read rows × cols space-separated integers for matrix1.
- Next, it will display "Elements of matrix 2: "and read rows × cols space-separated integers for matrix2.

Output Format:

- The program will print "Addition of matrices:" followed by the result matrix, with each row on a new line and elements separated by spaces ending with a space.

Note:

- The addition of two matrices can only be done when the dimensions of both matrices are the same, so we are taking the same dimensions for both matrices.
- Refer to the visible test cases for better understanding.

Source Code:

addTwoMatrices.c

```
#include<stdio.h>
int main(){
        int r,c;
        printf("Enter no of rows, columns: ");
        scanf("%d %d",&r,&c);
        int a[r][c], b[r][c],sum[r][c];
        printf("Elements of matrix 1: ");
        for(int i=0;i<r;i++){
                for(int j=0;j<c;j++){
                        scanf("%d",&a[i][j]);
                }
        }
        printf("Elements of matrix 2: ");
        for(int i=0;i<r;i++){
                for(int j=0;j<c;j++){
                        scanf("%d",&b[i][j]);
                }
        }
        for(int i=0;i<r;i++){
                for(int j=0;j<c;j++){
                        sum[i][j]=a[i][j]+b[i][j];
                }
        }
        printf("Addition of matrices:\n");
        for(int i=0;i<r;i++){
                for(int j=0;j<c;j++){
                        sum[i][j]=a[i][j]+b[i][j];
                        printf("%d ",sum[i][j]);
                }
                printf("\n");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of rows, columns:
1 2
Elements of matrix 1:
1 2

Elements of matrix 2:
9 8
Addition of matrices:
10 10

Test Case - 2
User Output
Enter no of rows, columns:
2 3
Elements of matrix 1:
1 2 3 4 5 6
Elements of matrix 2:
9 8 7 6 5 4
Addition of matrices:
10 10 10
10 10 10

S.No: 32	Exp. Name: ***Concatenate two given strings without using string library functions***	**Date: 2025-12-27**

Aim:

Write a program to `concatenate` two given strings without using string library functions.

At the time of execution, the program should print the message on the console as:

```
string1 :
```

For example, if the user gives the **input** as:

```
string1 : ILove
```

Next, the program should print the message on the console as:

```
string2 :
```

For example, if the user gives the **input** as:

```
string2 : Coding
```

then the program should **print** the result as:

```
concatenated string = ILoveCoding
```

Note: Do use the **printf()** function with a **newline** character (`\n`) at the end.

Source Code:

Program605.c

```
#include<stdio.h>
int main(){
        char str1[100],str2[100];
        int i=0,j=0;
        printf("string1 : ");
        scanf("%s",str1);
        printf("string2 : ");
        scanf("%s",str2);
        while(str1[i]!='\0'){
                i++;
        }
        while(str2[j]!='\0'){
                str1[i]=str2[j];
                i++;
                j++;
        }
        str1[i]='\0';
        printf("concatenated string = %s\n",str1);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
string1 :
ILove
string2 :
Coding
concatenated string = ILoveCoding

Test Case - 2
User Output
string1 :
1234
string2 :
567
concatenated string = 1234567

S.No: 39	Exp. Name: ***Write a C Program to store information using Structures with DMA***	**Date: 2025-12-27**

Aim:

Write a program to create a **list of nodes** using self-referential structure and print that data.

At the time of execution, the program should print the message on the console as:

```
Enter an integer value :
```

For example, if the user gives the **input** as:

```
Enter an integer value : 10
```

Next, the program should print the message on the console as:

```
Do u want another list (y|n) :
```

if the user gives the **input** as:

```
Do u want another list (y|n) : y
```

The input to the list is continued up to the user says n (No)

For example, if the user gives the **input** as:

```
Enter an integer value : 20
Do u want another list (y|n) : y
Enter an integer value : 30
Do u want another list (y|n) : n
```

Finally, the program should print the result on the console as:

```
The elements in the single linked lists are : 10-->20-->30-->NULL
```

Note: Write the functions **create()** and **display()** in CreateNodes.c.

Source Code:

StructuresWithDma.c

```
#include <stdio.h>
#include <stdlib.h>
struct list {
        int data;
        struct list *next;
};
#include "CreateNodes.c"
void main() {
        struct list *first = NULL;
        first = create(first);
        printf("The elements in the single linked lists are : ");
        display(first);
}
```

CreateNodes.c

```
struct list* create(struct list *first) {
        char op;
        struct list *q, *temp;
        do {
         temp =(struct list*)malloc(sizeof(struct list));
                printf("Enter an integer value : ");
                scanf("%d", &temp->data );
                temp->next = NULL;
                if (first == NULL) {
                        first = temp;
                } else {
                        q->next = temp;
                }
                q = temp;
                printf("Do u want another list (y|n) : ");
                scanf(" %c", &op);
        } while(op == 'y' || op == 'Y');
        return first;
}
void display(struct list *first) {
        struct list *temp = first;
        while (temp!=NULL ) {
                printf("%d-->", temp->data);
                temp = temp->next;
        }
        printf("NULL\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an integer value :
10
Do u want another list (y|n) :
y
Enter an integer value :
20
Do u want another list (y|n) :
y
Enter an integer value :

30
Do u want another list (y|n) :
n
The elements in the single linked lists are : 10-->20-->30-->NULL

S.No: 43	Exp. Name: ***Write a C Program to find nCr using Factorial recursive function***	**Date: 2025-12-27**

Aim:

Draw the flowchart and write a recursive **C** function to find the factorial of a number, `n!`, defined by **fact(n) = 1**, if n = 0. Otherwise **fact(n) = n * fact(n-1)**.

Using this function, write a **C** program to compute the binomial coefficient n_{C_r}. Tabulate the results for different values of **n** and **r** with suitable messages.

At the time of execution, the program should print the message on the console as:

```
Enter the values of n and r :
```

For example, if the user gives the **input** as:

```
Enter the values of n and r : 4 2
```

then the program should **print** the result as:

```
The value of 4c2 = 6
```

If the input is given as `2` and `5` then the program should print the result as:

```
Enter valid input data
```

Note: Write the recursive function **factorial()** in `Lab14a.c`.

Source Code:

Lab14a.c

```
#include<stdio.h>
int factorial(int n){
        if(n==0)
                return 1;
        return n*factorial(n-1);
}
```

Lab14.c

```
#include <stdio.h>
#include "Lab14a.c"
void main() {
        int n, r;
        printf("Enter the values of n and r : ");
        scanf("%d %d", &n, &r);
        if (n >= r)
                printf("The value of %dc%d = %d\n", n, r, factorial(n)
/ (factorial(r) * factorial(n - r)));
        else
                printf("Enter valid input data\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the values of n and r :
10 4
The value of 10c4 = 210

Test Case - 2
User Output
Enter the values of n and r :
7 9
Enter valid input data

Test Case - 3
User Output
Enter the values of n and r :
5 2
The value of 5c2 = 10

S.No: 44	Exp. Name: ***Write a Program to find the Length of a String***	**Date: 2025-12-27**

Aim:

Write a **C** program to find the `length` of a given string.

Sample Input and Output - 1:

```
Enter the string : CodeTantra
Length of CodeTantra : 10
```

Source Code:

StrLength.c

```
#include <stdio.h>
#include "StrLength1.c"
void main() {
        char str[30];
        printf("Enter the string : ");
        scanf("%s", str);
        printf("Length of %s : %d\n", str, myStrLen(str));
}
```

StrLength1.c

```
#include<stdio.h>
int myStrLen(char str[]){
        int len=0;
        while(str[len]!='\0'){
                len++;

        }
        return len;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

```
Enter the string :
CodeTantra
Length of CodeTantra : 10
```

Test Case - 2

User Output

```
Enter the string :
IndoUsUk
Length of IndoUsUk : 8
```

Test Case - 3

User Output

```
Enter the string :
MalayalaM
Length of MalayalaM : 9
```

Test Case - 4

User Output

```
Enter the string :
Oh!MyGod
Length of Oh!MyGod : 8
```

S.No: 47	Exp. Name: ***Fibonacci series up to the given number of terms using Recursion***	**Date: 2025-12-27**

Aim:

Write a program to display the fibonacci series up to the given number of terms using recursion process.

Source Code:

fibonacciSeries.c

```
#include <stdio.h>
#include "fibonacciSeriesa.c"
void main() {
        int n, i;
        printf("n: ");
        scanf("%d", &n);
        printf("%d terms: ", n);
        for (i = 0; i < n; i++) {
                printf("%d ", fib(i));
        }
}
```

fibonacciSeriesa.c

```
// Complete the function fib()....
int fib(int n){
        if(n==0)
        return 0;
        if(n==1)
                return 1;
        return fib(n-1)+fib(n-2);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
n:
4
4 terms: 0 1 1 2

Test Case - 2
User Output
n:
10
10 terms: 0 1 1 2 3 5 8 13 21 34

S.No: 48	Exp. Name: ***LCM of two numbers using recursion***	**Date: 2025-12-27**

Aim:

Write a program to find the `lcm` (Least Common Multiple) of a given two numbers using recursion process.

The least common multiple (`lcm`) of two or more integers, is the smallest positive integer that is divisible by both a and b.

At the time of execution, the program should print the message on the console as:

```
Enter two integer values :
```

For example, if the user gives the **input** as:

```
Enter two integer values : 25 15
```

then the program should **print** the result as:

```
The lcm of two numbers 25 and 15 = 75
```

Note: Write the function **lcm()** and recursive function **gcd()** in `Program907a.c`.

Source Code:

Program907.c

```
#include <stdio.h>
#include "Program907a.c"
void main() {
        int a, b;
        printf("Enter two integer values : ");
        scanf("%d %d", &a, &b);
        printf("The lcm of two numbers %d and %d = %d\n", a, b, lcm(a,
b));
}
```

Program907a.c

```
#include<stdio.h>
int gcd(int a, int b){
        if(b==0){
                return a;
        }
        else
                return gcd(b, a%b);
}
int lcm(int a, int b){
        return (a*b)/gcd(a,b);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter two integer values :
34 24
The lcm of two numbers 34 and 24 = 408

Test Case - 2
User Output
Enter two integer values :
6 9
The lcm of two numbers 6 and 9 = 18

Test Case - 3
User Output
Enter two integer values :
345 467
The lcm of two numbers 345 and 467 = 161115

Test Case - 4
User Output

Enter two integer values :
100 88
The lcm of two numbers 100 and 88 = 2200

Test Case - 5
User Output
Enter two integer values :
123 420
The lcm of two numbers 123 and 420 = 17220

S.No: 49	Exp. Name: ***Write a C program to find the Factorial of a given number using Recursion***	**Date: 2025-12-27**

Aim:

Write a program to find the `factorial` of a given number using recursion process.

Note: Write the recursive function **factorial()** in `Program901a.c`.

Source Code:

Program901.c

```
#include <stdio.h>
#include "Program901a.c"
void main() {
        long int n;
        printf("Enter an integer : ");
        scanf("%ld", &n);
        printf("Factorial of %ld is : %ld\n", n ,factorial(n));
}
```

Program901a.c

```
#include<stdio.h>
long int factorial(long int n){
        if(n==0 || n==1)
                return 1;
        else
                return n*factorial(n-1);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an integer :
5
Factorial of 5 is : 120

Test Case - 2
User Output
Enter an integer :
4
Factorial of 4 is : 24

Test Case - 3
User Output
Enter an integer :
8
Factorial of 8 is : 40320

Test Case - 4
User Output
Enter an integer :
0
Factorial of 0 is : 1

S.No: 50	Exp. Name: ***Ackermann function using Recursion***	**Date: 2025-12-27**

Aim:

Write a program to implement Ackermann function using recursion process.

At the time of execution, the program should print the message on the console as:

```
Enter two numbers :
```

For example, if the user gives the **input** as:

```
Enter two numbers : 2 1
```

then the program should **print** the result as:

```
A(2, 1) = 5
```

Source Code:

AckermannFunction.c

```
#include <stdio.h>
#include "AckermannFunction1.c"
void main() {
        long long int m, n;
        printf("Enter two numbers : ");
        scanf("%lli %lli", &m, &n);
        printf("A(%lli, %lli) = %lli\n", m, n, ackermannFun(m, n));
}
```

AckermannFunction1.c

```
#include<stdio.h>
long long int ackermannFun(long long int m, long long int n){
        if(m==0)
                return n+1;
        else if(m>0 && n==0)
                return ackermannFun(m-1,1);
        else
                return ackermannFun(m-1,ackermannFun(m,n-1));
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter two numbers :
0 1
A(0, 1) = 2

Test Case - 2
User Output
Enter two numbers :
2 2
A(2, 2) = 7

Test Case - 3
User Output
Enter two numbers :
2 1
A(2, 1) = 5

Test Case - 4
User Output
Enter two numbers :
1 1
A(1, 1) = 3

Test Case - 5
User Output
Enter two numbers :
1 0
A(1, 0) = 2

Test Case - 6

User Output
Enter two numbers :
2 3
A(2, 3) = 9

S.No: 51	Exp. Name: ***Problem solving***	Date: 2025-12-27

Aim:

Write a program to find the `sum` of `n` natural numbers using recursion process.

At the time of execution, the program should print the message on the console as:

```
Enter value of n :
```

For example, if the user gives the **input** as:

```
Enter value of n : 6
```

then the program should **print** the result as:

```
Sum of 6 natural numbers = 21
```

Note: Write the recursive function **sum()** in `Program903a.c`.

Source Code:

Program903.c

```
#include <stdio.h>
#include "Program903a.c"
void main() {
        int n;
        printf("Enter value of n : ");
        scanf("%d", &n);
        printf("Sum of %d natural numbers = %d\n", n, sum(n));
}
```

Program903a.c

```
#include<stdio.h>
int sum(int n){
        if(n==0)
                return 0;
        else
                return n+sum(n-1);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter value of n :
5
Sum of 5 natural numbers = 15

Test Case - 2
User Output
Enter value of n :
9
Sum of 9 natural numbers = 45